ChainLight

Biconomy

Biconomy Token Paymaster Security Audit

: Token Paymaster

Dec 2, 2024

Revision 1.0

ChainLight@Theori

Table of Contents

Executive Summary

Starting on Nov 11th, 2024, ChainLight of Theori audited the smart contract of Biconomy Token Paymaster for one week. In the audit, we primarily considered the issues/impacts listed below.

- Theft of funds
- ERC-7562 violation
- Functionality failure

As a result, we identified issues as listed below.

- Total: 6
- High: 1 (Theft of funds)
- Medium: 3 (ERC-7562 violation, Functionality failure)
- Informational: 2

Audit Overview

Scope

Name	Biconomy Token Paymaster Security Audit
Target / Version	• Git Repository (`bcnmy/.../BiconomyTokenPaymaster.sol`): commit `74de3c5c7909dcab11f2c01f2b9275cf1608b657`
Application Type	Smart contracts
Lang. / Platforms	Smart contracts [Solidity]

Code Revision

N/A

Severity Categories

Severity	Description
Critical	The attack cost is low (not requiring much time or effort to succeed in the actual attack), and the vulnerability causes a high-impact issue. (e.g., Effect on service availability, Attacker taking financial gain)
High	An attacker can succeed in an attack which clearly causes problems in the service's operation. Even when the attack cost is high, the severity of the issue is considered "high" if the impact of the attack is remarkably high.
Medium	An attacker may perform an unintended action in the service, and the action may impact service operation. However, there are some restrictions for the actual attack to succeed.
Low	An attacker can perform an unintended action in the service, but the action does not cause significant impact or the success rate of the attack is remarkably low.
Informational	Any informational findings that do not directly impact the user or the protocol.
Note	Neutral information about the target that is not directly related to the project's safety and security.

Status Categories

Status	Description
Reported	ChainLight reported the issue to the client.
WIP	The client is working on the patch.
Patched	The client fully resolved the issue by patching the root cause.
Mitigated	The client resolved the issue by reducing the risk to an acceptable level by introducing mitigations.
Acknowledged	The client acknowledged the potential risk, but they will resolve it later.
Won't Fix	The client acknowledged the potential risk, but they decided to accept the risk.

Finding Breakdown by Severity

Category	Count	Findings
Critical	**0**	• N/A
High	**1**	• `BTPMQ4-002`
Medium	**3**	• `BTPMQ4-001` • `BTPMQ4-003` • `BTPMQ4-005`
Low	**0**	• N/A
Informational	**2**	• `BTPMQ4-004` • `BTPMQ4-006`
Note	**0**	• N/A

Findings

Summary

#	ID	Title	Severity	Status
1	BTPMQ4-001	`Uniswapper._setTokenPool()` must use `forceApprove()` instead of `approve()`	Medium	Patched
2	BTPMQ4-002	In `_validatePaymasterUserOp()`, `priceMarkup` should be applied to `maxPenalty` when calculating the `prechargedAmount`	High	Patched
3	BTPMQ4-003	`BiconomyTokenPaymaster` must implement `receive()`	Medium	Patched
4	BTPMQ4-004	In `_validatePaymasterUserOp()`, an incorrect `tokenPrice` may be provided in `EXTERNAL` mode	Informational	Acknowledged
5	BTPMQ4-005	To comply with ERC-7562, `block.timestamp` should not be used during the validation phase	Medium	Won't Fix
6	BTPMQ4-006	Minor Suggestions	Informational	Patched

#1 `BTPMQ4-001` `Uniswapper._setTokenPool()` must use `forceApprove()` instead of `approve()`

ID	Summary	Severity
`BTPMQ4-001`	For certain tokens that do not fully comply with the ERC20 standard, such as USDT, calling `approve()` multiple times may result in a revert.	Medium

Description

When changing the `tokenToPools` mapping for an already set token in `Uniswapper._setTokenPool()`, if the token is one like USDT that can only be `approve()` when the current `allowance` is zero, it will always revert.

Impact

Medium

When the paymaster is deployed, it calls approve twice for the tokens in swappableTokens. As a result, if tokens like USDT are included in `swappableTokens`, the contract may fail to deploy properly. Additionally, for tokens like USDT, `updateSwappableTokens()` cannot be called more than once. This means that multiple `poolFeeTiers` cannot be set for such tokens.

Recommendation

It is recommended to use OpenZeppelin's `SafeERC20.forceApprove()` instead of `approve()`.

Remediation

Patched

The issue has been resolved as recommended.

#2 `BTPMQ4-002` In `_validatePaymasterUserOp()`, `priceMarkup` should be applied to `maxPenalty` when calculating the `prechargedAmount`

ID	Summary	Severity
`BTPMQ4-002`	The paymaster must exclude the `maxPenalty` when calculating the `prechargedAmount`.	High

Description

In `_validatePaymasterUserOp()`, when calculating the `prechargedAmount` value included in the `context`, it is necessary to apply `priceMarkup` to `maxPenalty` in a manner consistent with the calculation of `tokenAmount`. The paymaster should receive from `userOp.sender` the token amount with `priceMarkup` applied for the penalty as well. However, if `priceMarkup` is not applied when calculating the `prechargedAmount` value as currently implemented, an additional refund of maxPenalty * tokenPrice * (externalPriceMarkup - _PRICE_DENOMINATOR) / (1e18 * _PRICE_DENOMINATOR) tokens will occur in `postOp()`. This means the paymaster is receiving from `userOp.sender` the token amount without `priceMarkup` applied for the potential penalty.

Impact

High

The paymaster might incur a loss if a penalty occurs because the refund in `postOp()` is performed including `maxPenalty`.

Recommendation

When calculating the `prechargedAmount` value, in `EXTERNAL` mode, it should be calculated as tokenAmount - ((maxPenalty * tokenPrice * externalPriceMarkup) / (1e18 * _PRICE_DENOMINATOR)), and in `INDEPENDENT` mode, it should be calculated as tokenAmount - ((maxPenalty * tokenPrice * independentPriceMarkup) / (1e18 * _PRICE_DENOMINATOR)).

Remediation

Patched

The issue has been resolved as recommended.

#3 `BTPMQ4-003` `BiconomyTokenPaymaster` must implement `receive()`

ID	Summary	Severity
`BTPMQ4-003`	The `BiconomyTokenPaymaster` contract does not implement the `receive()`.	Medium

Description

`BiconomyTokenPaymaster` does not implement `receive()`, so it cannot receive native tokens. Therefore, `swapTokenAndDeposit()` will always revert at `_unwrapWeth()`.

Impact

Medium

The `swapTokenAndDeposit()` call always triggers a revert.

Recommendation

It is recommended to implement `receive()` to enable the contract to receive native tokens. Additionally, to prevent accidental sending of native tokens, you could add an Access Control List (ACL) to `receive()` so that only the Uniswap v3 Router's `WETH9` address can send native tokens, but this would incur additional gas costs.

Remediation

Patched

The issue has been resolved as recommended.

#4 `BTPMQ4-004` In `_validatePaymasterUserOp()`, an incorrect `tokenPrice` may be provided in `EXTERNAL` mode

ID	Summary	Severity
`BTPMQ4-004`	In `validatePaymasterUserOp()`, the comment for the price in EXTERNAL mode contains an incorrect formula.	Informational

Description

In `_validatePaymasterUserOp()`, it is specified that in `EXTERNAL` mode, `tokenPrice` is token / native * 10 ** token decimals. `tokenPrice` must be similar to `_getPrice()`, i.e., (nativeTokenPrice * 10 ** token decimals) / tokenPrice.

Impact

Informational

Recommendation

The comment for `tokenPrice` needs to be clarified and updated.

Remediation

Acknowledged

The team mentioned that the token price is being calculated correctly on the backend, as recommended in the issue.

#5 `BTPMQ4-005` To comply with ERC-7562, `block.timestamp` should not be used during the validation phase

ID	Summary	Severity
`BTPMQ4-005`	In the validation phase, blocked opcodes defined by ERC-7562 cannot be used. The `_fetchPrice()`, which is called during the validation phase, uses `block.timestamp`, thereby violating ERC-7562.	Medium

Description

In the `validatePaymasterUserOp()` of `BiconomyTokenPaymaster`, the token price is fetched from the oracle in `INDEPENDENT` mode. However, in the `_fetchPrice()`, `block.timestamp` is used to verify when the oracle's price was last updated. According to ERC-7562, using the `TIMESTAMP` opcode in the validation phase is prohibited. If blocked opcodes are accessed during the validation phase, bundlers will discard the corresponding userOperation. Therefore, the structure should be modified to fetch the token price from the oracle during the execution phase rather than the validation phase.

Impact

Medium

Since `validatePaymasterUserOp()` uses blocked opcodes, bundlers discard the userOperations that use the biconomy token paymaster, preventing the userOp from being executed successfully.

Recommendation

It is recommended to modify the token price update structure so that token price updates through the oracle are executed in the `postOp` phase, while the validation phase references a storage variable containing the latest token price. If `_fetchPrice()` is not called for an extended period, leading to a significant discrepancy between the token price stored in the storage variable and the actual token price, an off-chain bot can directly call `fetchPrice()`.

Remediation

Won't Fix

The team mentioned that they will not fix this issue because they plan to use a separate alt mempool that relaxes the restrictions on the `blocktimestamp` opcode.

#6 BTPMQ4-006 Minor Suggestions

ID	Summary	Severity
BTPMQ4-006	The description includes multiple suggestions for preventing incorrect settings caused by operational mistakes, mitigating potential issues, improving code maturity and readability, and other minor issues.	Informational

Description

- It is recommended to add a `removeTokenDirectory()` to allow the removal of supported tokens.
- In `Uniswapper.constructor()`, `approve()` is performed for each token. In the `BiconomyTokenPaymaster.constructor()` that inherits from it, `approve()` is also performed for each token. If there is a token like USDT in the token list that can only be `approve()`d when the current `allowance` is zero, the contract deployment will always revert.
- In the comments for `setPriceExpiryDuration`, the explanations for `@dev` and `@param` are incorrect.
- If the native token's decimal is not 18, incorrect values will be calculated for `tokenAmount` in `_validatePaymasterUserOp()`. It is recommended to use a variable like `nativeAssetDecimal` instead of the constant `1e18` in the formula, and set its value separately in the `constructor()`.
- For penalty refunds, it is recommended to add `tokenPrice` to `PaidGasInTokens`. On the backend, it is recommended to refund (maxPenalty - realPenalty) * (tokenPrice in event * priceMarkup) / (1e18 * _PRICE_DENOMINATOR) amount of tokens.
- In `EXTERNAL` mode and `INDEPENDENT` mode, the types of the variables `tokenPrice` and `priceMarkup` differ. It is recommended to standardize them as `uint256`. Also, in `_fetchPrice()`, it is recommended not to typecast `uint256` to `uint192` when returning.
- `swapTokenAndDeposit()`, `withdrawERC20()`, and `withdrawERC20Full()` should consider the refund amount for the penalty when executing.
- Since the constant `_SWAP_PRICE_DENOMINATOR` in `Uniswapper` is unused, it is recommended to delete it.
- It is recommended to add events for `swapTokenAndDeposit()` and `updateSwappableTokens()`.

- It is recommended to add the condition require(block.timestamp >= newPriceExpiryDuration) in `constructor()` and `setPriceExpiryDuration()`.
- In `_validatePaymasterUserOp()`, it is recommended to revert if the return value of `_getPrice()` is zero in `INDEPENDENT` mode.
- In `_fetchPrice()`, it is better to verify `priceExpiryDuration` differently for each token since, with Chainlink, the price update interval varies by token.
- In the comments for `setSigner()`, `_newVerifyingSigner` should be corrected to `newVerifyingSigner`.
- In the comments for `setSigner()`, `VerifyingSignerChanged` should be corrected to `UpdatedVerifyingSigner`.

Impact

Informational

Recommendation

Consider applying the suggestions in the description above.

Remediation

Patched

The issues have been resolved as recommended.

Revision History

Version	Date	Description
1.0	Dec 2, 2024	Initial version

Theori, Inc. (“We”) is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.